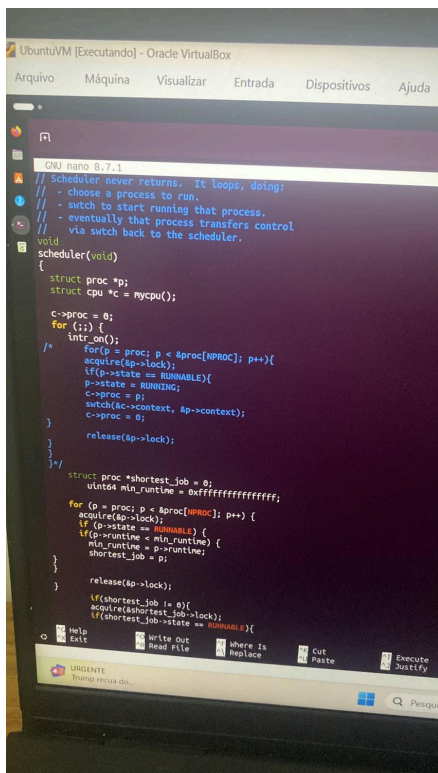


Relatório de Implementação: Escalonador SJF e Métrica de Tempo de Execução no xv6

Este relatório apresenta as modificações estruturais realizadas no sistema operacional xv6 para a substituição do algoritmo de escalonamento padrão por uma abordagem baseada em Shortest Job First (SJF). Para dar suporte a essa lógica, foi desenvolvida uma nova chamada de sistema (syscall) denominada getruntime, projetada para contabilizar e retornar o tempo de processamento consumido por cada processo individual. A seguir, é detalhada de forma cronológica as etapas de implementação, mapeamento no kernel e validação prática do sistema em ambiente emulado.



Inicialmente, a primeira parte da função scheduler é modificada. As variáveis `min_runtime` e `shortest_job` são inicializadas e um laço for é executado para percorrer a tabela de processos e encontrar qual processo no estado `RUNNABLE` possui o menor tempo de execução acumulad

```

    p->state = RUNNING;
    c->proc = p;
    switch(&c->context, &p->context);
    c->proc = 0;

    release(&p->lock);
}
}
}*/

struct proc *shortest_job = 0;
uint64 min_runtime = 0xffffffffffffffff;

for (p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);
    if (p->state == RUNNABLE) {
        if (p->runtime < min_runtime) {
            min_runtime = p->runtime;
            shortest_job = p;
        }
    }
    release(&p->lock);
}

if (shortest_job != 0) {
    acquire(&shortest_job->lock);
    if (shortest_job->state == RUNNABLE) {
        shortest_job->state = RUNNING;
        c->proc = shortest_job;

        switch(&c->context, &shortest_job->context);
        c->proc = 0;

        release(&shortest_job->lock);
    }
}

void
sched(void)
{
}

```

Aqui é apresentada a continuação das alterações feitas na função scheduler. A busca é finalizada e a lógica de troca de contexto é inserida: o estado do processo escolhido (shortest_job) é alterado para RUNNING e a função switch é chamada para colocá-lo em execução na CPU.

```

    acquire(&tickslock);
    xticks = ticks;
    release(&tickslock);
    return xticks;
}

uint64
sys_getruntime(void)
{
    return myproc()->runtime;
}

```

A função sys_getruntime(void) é criada diretamente no código do kernel. Nela, a chamada myproc() é colocada para obter o processo atual e o valor da sua variável ->runtime é simplesmente retornado.

```

GNU nano 8.7.1
#define SBRK_ERROR ((char *)-1)

struct stat;

system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int *);
int pipe(int *);
int write(int, const void *, int);
int read(int, void *, int);
int close(int);
int kill(int);
int exec(const char *, char **);
int open(const char *, int);
int mknod(const char *, short, short);
int unlink(const char *);
int fstat(int fd, struct stat *);
int link(const char *, const char *);
int mkdir(const char *);
int chdir(const char *);
int dup(int);
int getpid(void);
char *sys_sbrk(int, int);
int pause(int);
int uptime(void);
int sync(void);
int getruntime(void);

// ulib.c
int stat(const char *, struct stat *);
char *strcpy(char *, const char *);
void *memcpy(void *, const void *, int);
char *strchr(const char *, char c);
int strcmp(const char *, const char *);
char *strcpy(const char *, const char *);
char *strcpy(const char *, const char *);
uint strlen(const char *);
void *memset(void *, int, uint);
int atoi(const char *);
void *memcpy(void *, const void *, uint);
char *sbrk(int);

```

No arquivo de cabeçalho user.h, a declaração `int getruntime(void);` é digitada no final da lista para liberar essa chamada de sistema para os programas de usuário.

```

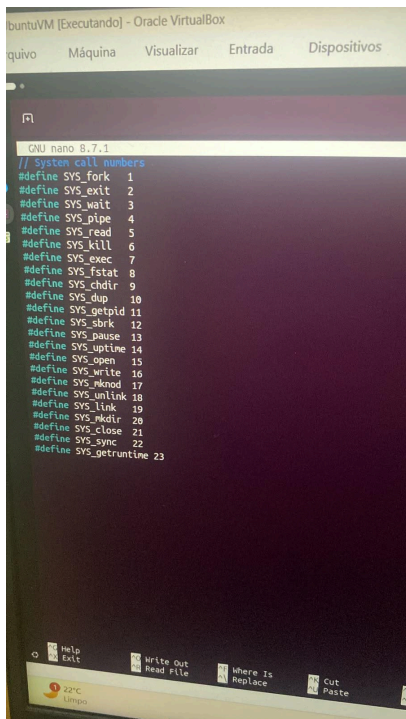
GNU nano 8.7.1
print "#include \"kernel/syscall.h\"\\n";

sub entry {
    my $prefix = "sys_";
    my $name = shift;
    if ($name eq "sbrk") {
        print ".global $prefix$name\\n";
        print "$prefix$name\\n";
    } else {
        print ".global $name\\n";
        print "$name\\n";
    }
    print ".li a7, sys_$name\\n";
    print ".ecall\\n";
    print ".ret\\n";
}

entry("fork");
entry("exit");
entry("wait");
entry("pipe");
entry("read");
entry("write");
entry("close");
entry("kill");
entry("exec");
entry("open");
entry("mknod");
entry("unlink");
entry("fstat");
entry("link");
entry("mkdir");
entry("chdir");
entry("dup");
entry("getpid");
entry("sbrk");
entry("pause");
entry("uptime");
entry("sync");
entry("getruntime");

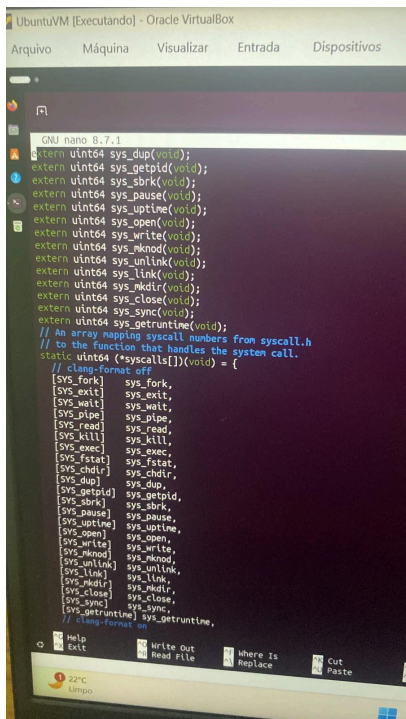
```

O script `usys.pl` é aberto e a linha `entry("getruntime");` é colada no final da lista para que o código assembly da syscall seja gerado automaticamente.



```
GNU nano 8.7.1
// syscall numbers
#define SYS_fork 1
#define SYS_exit 2
#define SYS_wait 3
#define SYS_pipe 4
#define SYS_read 5
#define SYS_kill 6
#define SYS_exec 7
#define SYS_fstat 8
#define SYS_chdir 9
#define SYS_dup 10
#define SYS_getpid 11
#define SYS_sbrk 12
#define SYS_pause 13
#define SYS_uptime 14
#define SYS_open 15
#define SYS_write 16
#define SYS_mknod 17
#define SYS_unlink 18
#define SYS_link 19
#define SYS_mkdir 20
#define SYS_close 21
#define SYS_sync 22
#define SYS_getruntime 23
```

No arquivo syscall.h, o número da nova chamada de sistema é definido. A linha `#define SYS_getruntime 23` é colada no final do código.



```
GNU nano 8.7.1
extern uint64 sys_dup(void);
extern uint64 sys_getpid(void);
extern uint64 sys_sbrk(void);
extern uint64 sys_pause(void);
extern uint64 sys_uptime(void);
extern uint64 sys_open(void);
extern uint64 sys_write(void);
extern uint64 sys_mknod(void);
extern uint64 sys_unlink(void);
extern uint64 sys_link(void);
extern uint64 sys_mkdir(void);
extern uint64 sys_close(void);
extern uint64 sys_sync(void);
extern uint64 sys_getruntime(void);
// An array mapping syscall numbers from syscall.h
// to the function that handles the system call.
static uint64 (*syscalls[])(void) = {
    // clang-format off
    [SYS_fork] sys_fork,
    [SYS_exit] sys_exit,
    [SYS_wait] sys_wait,
    [SYS_pipe] sys_pipe,
    [SYS_read] sys_read,
    [SYS_kill] sys_kill,
    [SYS_exec] sys_exec,
    [SYS_fstat] sys_fstat,
    [SYS_chdir] sys_chdir,
    [SYS_dup] sys_dup,
    [SYS_getpid] sys_getpid,
    [SYS_sbrk] sys_sbrk,
    [SYS_pause] sys_pause,
    [SYS_uptime] sys_uptime,
    [SYS_open] sys_open,
    [SYS_write] sys_write,
    [SYS_mknod] sys_mknod,
    [SYS_unlink] sys_unlink,
    [SYS_link] sys_link,
    [SYS_mkdir] sys_mkdir,
    [SYS_close] sys_close,
    [SYS_sync] sys_sync,
    [SYS_getruntime] sys_getruntime,
    // clang-format on
}
```

O arquivo syscall.c no kernel é modificado. Primeiro, a declaração externa `extern uint64 sys_getruntime(void);` é colada e, em seguida, o trecho `[SYS_getruntime] sys_getruntime,` é inserido no meio do array principal para mapear a função corretamente.

```
riscv64-linux-gnu-objdump -o user/_testesjf.o user/_testesjf.o
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_
n user/_dorphan user/_sync user/_testesjf
mmap 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) block
ballocc: first 1024 blocks have been allocated
ballocc: write bitmap block at sector 40
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -n 128M -img
display none

xv6 kernel is booting
hart 2 starting
hart 1 starting
Unit: starting sh
$ testesjf
--- Testando Syscall gettimeofday ---
Tempo inicial de CPU: 1 ticks
Tempo final apos o processamento: 3 ticks
$
```

Por fim, o sistema emulado é rodado no terminal e o programa de teste testesjf é executado. Como pode ser observado, o funcionamento ocorre corretamente e são impressos os resultados "Tempo inicial de CPU: 1 ticks" e depois "Tempo final apos o processamento: 3 ticks".